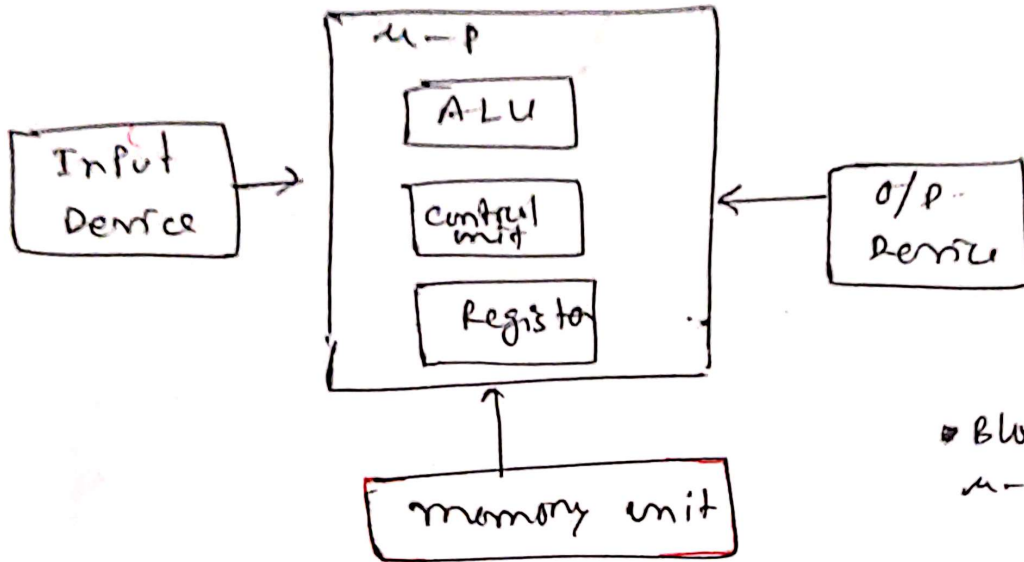


Pentium Microprocessor

Microprocessor vs Micro Controller :-



• Block diagram of μ -controller.

μ -P

- It contains only CPU
- Designer decides size of RAM, ROM, I/O handling
- General purpose
- Heart of the computer system
- High speed & high cost
- I/O, memory connect externally. So the circuit become large.

μ -controller

- CPU, Memory, Input, Output, Timer are on single chip
- Fixed size of RAM, ROM & I/O handling.
- Specific purpose.
- Heart of the embedded system.
- Relatively low cost and low speed.
- I/O and memory present internally, the circuit is small.

u-P

→ Power consumption is high

→ Difficult to replace

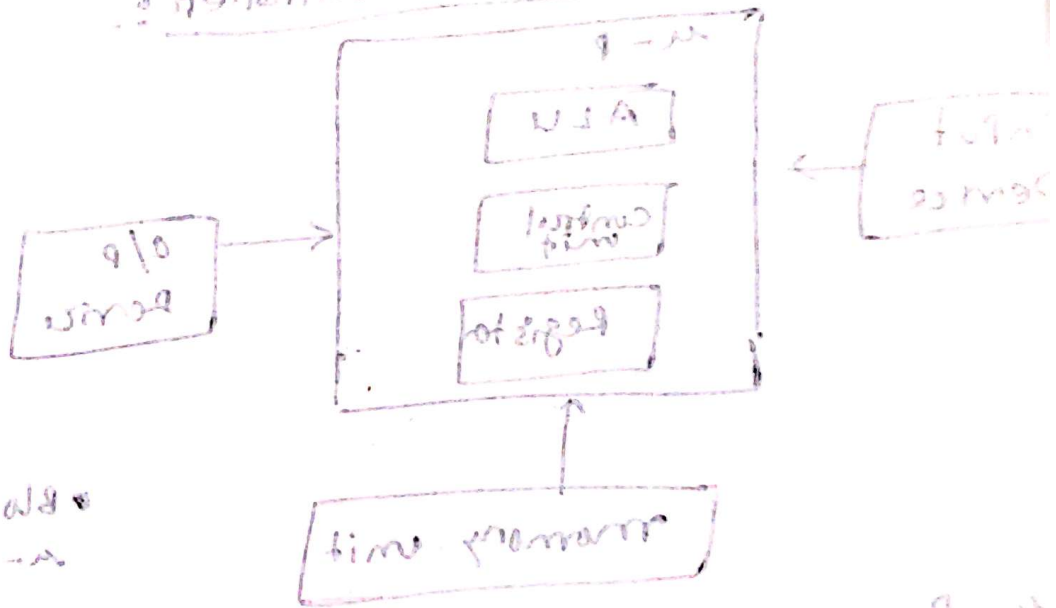
→ Ex:- 8085, 8086, Core i3, Core i5, Core i7 etc

u-computer

→ power consumption is low

→ Easy to replace

→ Ex:- 8051, AVR, PIC



Block diagram of a microcontroller

Microcontroller

Input, output, memory, ALU - Timer are on single chip

Fixed size of ROM, RAM, I/O, timer, ALU, registers

Specific hardware

Hardwired logic

Relatively low cost and low power

Low power and low cost

Highly integrated circuit

Microcontroller can

perform all the functions of a microprocessor

of the computer

low cost & high speed

Low power consumption & high reliability

4-P. features

- ① Low cost : (IC)
- ② Less power dissipation : (MOS semiconductor)
- ③ Low power consumption (MOS technology)
- ④ High Speed
- ⑤ Versatile
- ⑥ Reliable
- ⑦ Portable
- ⑧ Small size

RISC

- ① Reduced Instruction Set Computer
- ② Large program
- ③ Consist of large number register
- ④ More RAM use
- ⑤ Software base
Ex: Sun's SPARC, PowerPC
- ⑥ Single clock system

CISC

- ① Complex instruction set computer
- ② Lower program
- ③ Less number of register
- ④ Less RAM use
- ⑤ Hardware base
Ex: Pentium process,
Intel 386, 486.
- ⑥ Multi clock system

8085 4-P

* 8 bit 4-P

* NMOS base

* 8 bit data bus, 16 bit address bus.

* 6 General Purpose Register, (8 bit)
(A, B, C, D, E, H, L)

* 40 Pin.

* 16 bit Program Counter

* 16 bit stack pointer.

(Push/Pop)

Program Counter:- 16 bit register used to store the memory address location of the next instruction to be executed.

stack pointer:- 16 bit register works like stack, which is always incremented/decremented by 2 during pop/push operations.
(push → incremented)
(pop → decremented)

Flag Register :-

D7	D6	D5	D4	D3	D2	D1	D0
S	Z	*	AC	*	P	*	CY

Sign (S) = 0 $\rightarrow$ result positive
 1 $\rightarrow$ result negative.

Parity $\rightarrow$ 1 all number of 1 is even
 0 " " " " " " " " odd.

Zero (Z) $\rightarrow$ If result is zero, zero flag is set (1)
 If " " " " " " " " not zero, " " reset (0)

AC $\rightarrow$ D3 $\rightarrow$ D4 6, 2nd carry 2nd 1.

CY $\rightarrow$ out of 7 bit 2nd 1.

```

0 1 0 0 0 0 0 1
0 1 0 0 0 0 0 0
-----
1 0 0 0 1 0 0 0
  
```

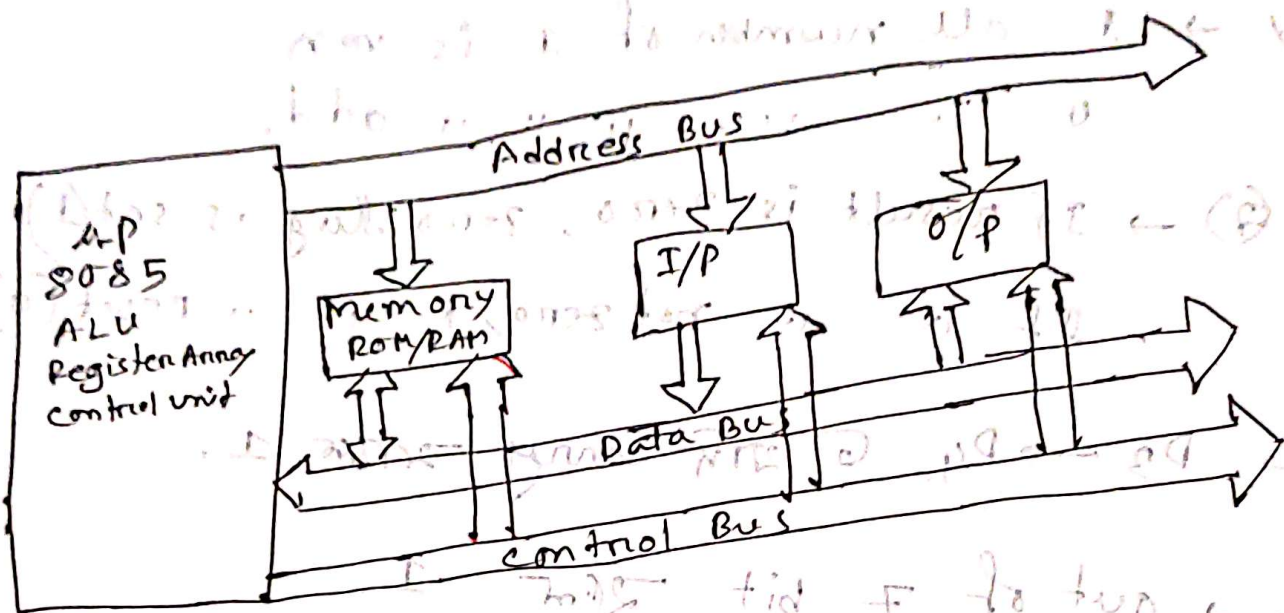
S = 0, Z = 0, AC = 1, P = 1, CY = 0

0 0 X 1 X 1 X 0

8086 → Flag register 16 bit (2 bit active flag)

XXXXOF DF IF TF S Z X AC X P X CY

Buses in 8085



Address Bus

* 16 bit (A₀ - A₁₅)

- * used to identify the address of memory and I/O peripherals
- * unidirectional,

Control Bus

* Four Control signal in 8085

* Memory Read, Memory write, I/O Read & I/O write.

* By it, μ -P control the operation of memory with respect to input and output devices

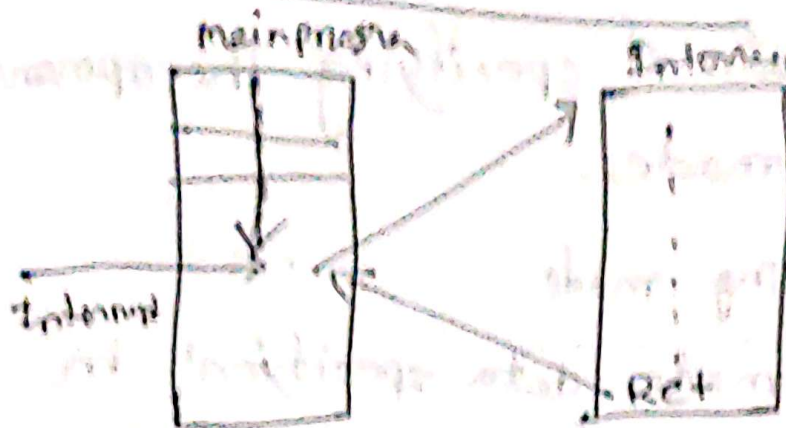
Data Bus

* Bidirectional

* Read operation direction is towards the μ -P. (ROM)

* For write operation direction is from μ -P.

Interrupt Service Routine



An interrupt is a special condition that makes the 8085 to execute ISR (Interrupt service routine).

The μ -P checks the interrupt during every instruction step:-

- When interrupt occurs the μ -P 8085 first complete current instruction.
- After that it pushes PC on the stack. (next instruction address copy mem)
- After that it disables interrupt flip flop so that processor cannot take interrupt until interrupt FF is enabled.
- Interrupt program execute $20H$ or $INTA$ flag or $RRST$ or call instruction call $20H$
- After that PC pop PC of $INTA$ μ -P main program execute $INTA$ or $INTF$ (Interrupt FF (INTF) active $20H$ (EI))

Addressing modes in 4-P 8085

* The various formats of specifying the operands are called addressing modes.

→ Immediate addressing mode

In this addressing mode, data specified in instruction is directly transferred into register.

EX:- `MVI C, 15H` ; 15H will get transferred to C.

→ Register Addressing mode

In this addressing mode, data is specified in register.

EX:-

`MOV A, C` ; C will get copied into A.

→ Direct addressing mode

In this addressing mode, address of operand is specified in instruction.

EX:- `LDA 1000H` ; A will get data from 1000H address.

→ Indirect Addressing mode

In this addressing mode, the address of operand is stored in register pair.

`LDA X B` (move content B-C register to the accumulator.)

Implied/Implicit Addressing mode

→ operand is hidden.

STC

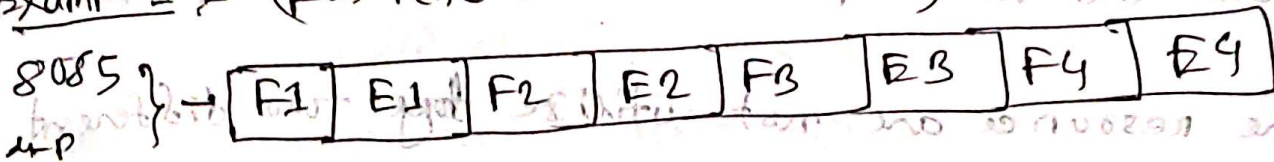
RPC

Pipelining

→ Pipeline defines the overlapping of stages for instruction execution.

→ Pipeline is done to improve execution speed of programs.

Example 1 → (F → Fetch, E → Execution)



structural Hazards in pipelining:-

→ Let's take four stage pipeline:

① Instruction fetch (IF)

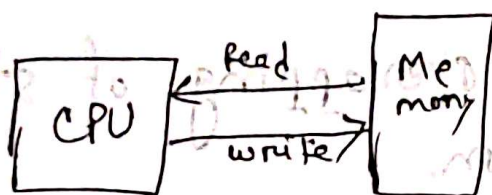
② Instruction decode + operand decode (ID)

③ Execute Instruction (EX)

④ Write back (WB)

Example of 4 stage pipeline.

	1	2	3	4	5	6	7
Memory Instruction 1	IF	ID	EX	WB			
2		IF	ID	EX	WB		
3			IF	ID	EX	WB	
4				IF	ID	EX	WB



same time 2 to 8 ms

WB means write operation from memory, IF means read from memory, structurally they are happening at a time but read and write operation at a time is not possible.

same resource are not utilize two different instruction.

	1	2	3	4	5	6	7	8
I ₁	IF	ID	EX	EX	EX	WB		
I ₂		IF	ID			EX	WB	

2 clock cycle are stalled

- Instruction decode (IF)
- Instruction fetch (IF)
- Execute Instruction (EX)
- Write back (WB)

Data Hazards in Pipelining :-

I_1 : ADD R_1, R_2, R_3

I_2 :- ADD R_4, R_1, R_5

	1	2	3	4	5	6	7	8
I_1	IF	ID	IE	m	WB (Result of R_1)			
I_2		IF	ID	EX	m	WB		

False result in R_4 .

stage 5 G first R_1 ka result update hoga ki second instruction G stage 3 me ($R_1 + R_5$) decode hoga so connect result jaisa hi and EX G false result ka

har stage ke har har buffer use krta hai. buffer register result hold krta hai, so buffer ka result jaisa hi execute krta hai hazard krta hai. solution is operand forwarding hai.

	1	2	3	4	5	6	7
I_1	IF	ID	EX	m	WB		
I_2		IF	ID	EX	m	WB	

correct result.

$\cap$ → dependence.
 FI → first instruction
 NI → Next instruction
 IO → input operand
 OO → output operand

→ RAW (Read After Write) (True dependency)

ADD R_1, R_2, R_3

ADD R_4, R_1, R_5

$$IO(NI) \cap OO(FI) \neq \emptyset$$

→ WAR (Write After Read) (Anti dependency)

ADD R_3, R_2, R_3

ADD R_2, R_1, R_3

$$IO(FI) \cap OO(NI) \neq \emptyset$$

→ WAW (Write After Write (output dependancy))

ADD (R1), R2, R3

ADD (R1), R4, R5
o/p dependancy.

$001111 \wedge 001111 \neq 0$

Control Hazards:

→ control Hazards in pipeline is happening due to branch instruction.

Address	Instruction	1	2	3	4	5	6	7
1000	I ₁ → Branch Inst ⁿ 2000	IF	ID	EX	WB			
1004	I ₂		IF	ID	EX	WB		
1008	I ₃			IF	ID	EX	WB	
...	...				IF	ID	EX	WB
2000	I _n					IF	ID	EX

flush pipeline.

I₁ starts execute stage but it's branch instruction execute stage so 2000 address will jump to 2000. but during the execution stage already IF ID pipeline is started so it's wrong of 2nd stage flush stage then 2000 address of instruction execute stage.

flush stage = EX stage - 1

Features of Pentium 4-P

- Pentium 4-P was available with 66 MHz - 99 MHz clock.
(Higher the clock gives higher speed of execution of program.)
- It has 64 bit data bus.
- It has two 32 bit MPU and ALU. Due to two execution unit it is referred as superscalar architecture.
- It has five stage pipelining.
- It has on chip floating point unit. Floating point unit supports 8 stage pipelining. (speed is super.)
- Pentium 4-P supports branch prediction algorithm.
(So no need to flush the memory for pipelining)
- It has on chip L1 cache memory.

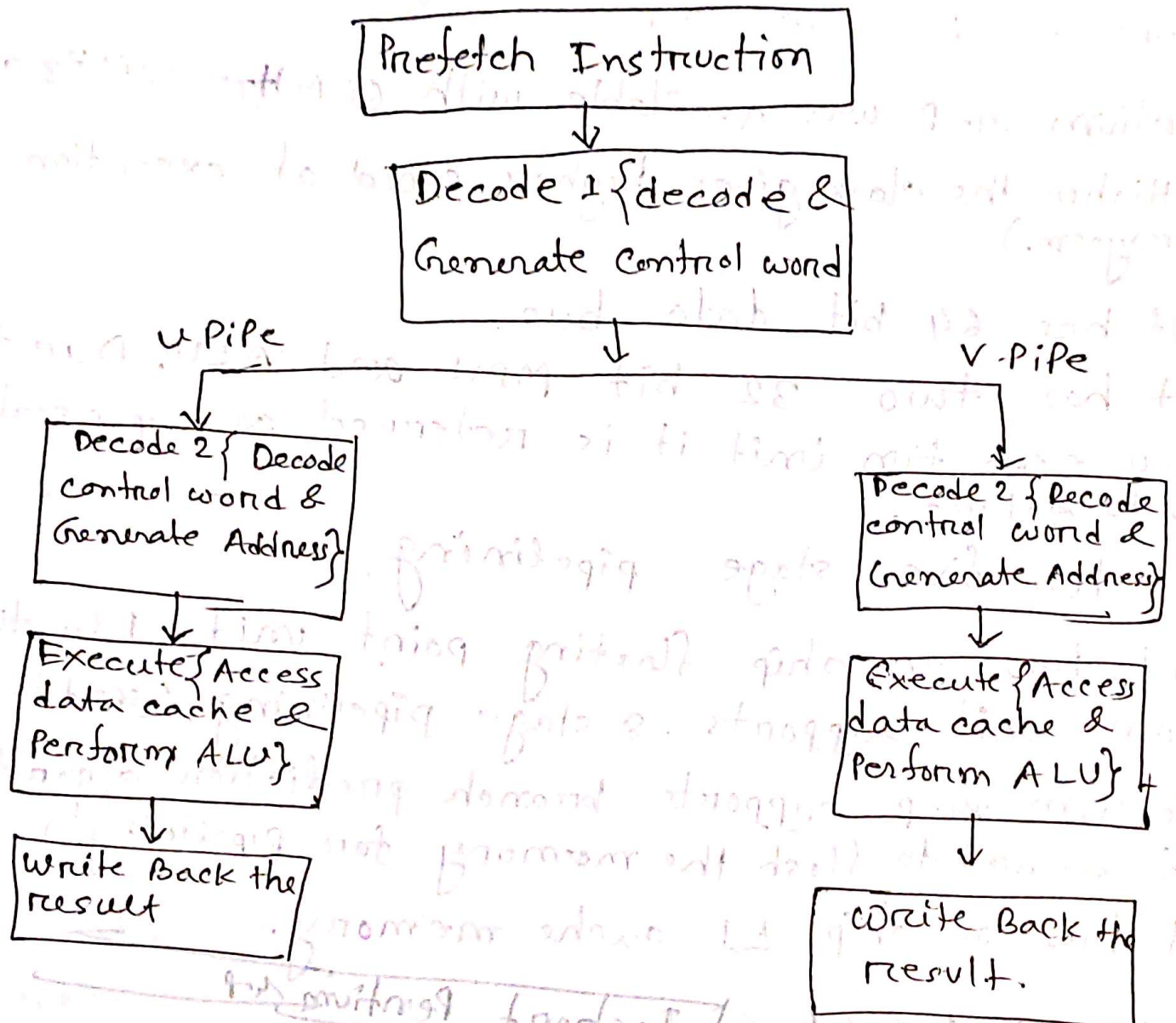
- supports branch prediction algorithm.
- supports integer UV pipelining.

Instant Pentium 4-P

- ① 64 bit u-P
- ② 64 bit data bus
- ③ 32 bit address bus
- ④ 5 stage pipelining
- ⑤ two 32 bit of MPU & ALU
- ⑥ supports floating point on chip. So 8 stage pipelining.
- ⑦ on chip L1 cache memory.

Pipeline

	IF	ID 1	ID 2	EX	WB
Instruction 1 u Pipe					
Instruction 2 v Pipe					



Handwritten notes at the bottom left, partially obscured and mirrored, include:

- 1st stage
- 2nd stage
- 3rd stage
- 4th stage
- 5th stage
- 6th stage
- 7th stage
- 8th stage
- 9th stage
- 10th stage
- 11th stage
- 12th stage
- 13th stage
- 14th stage
- 15th stage
- 16th stage
- 17th stage
- 18th stage
- 19th stage
- 20th stage

Handwritten notes at the bottom right, partially obscured and mirrored, include:

- 21st stage
- 22nd stage
- 23rd stage
- 24th stage
- 25th stage
- 26th stage
- 27th stage
- 28th stage
- 29th stage
- 30th stage
- 31st stage
- 32nd stage
- 33rd stage
- 34th stage
- 35th stage
- 36th stage
- 37th stage
- 38th stage
- 39th stage
- 40th stage

Comparison of i3, i5 & i7

* core means how much cpu can perform in parallel.
 i.e. core i3 means 3 cpu can perform in parallel.

Features

	i3	i5	i7
Number of core	* 4 for desktop * 4 for laptop	* 6 for desktop * 4 for laptop	* 8 for desktop * 6 for laptop
Hyper threading	* only dual core	* only dual core	* All core.

Hyper threading means a single core act as a dual cpu.
 for core i3 & core i5 they have dual core perform like that
 but for i7 all core perform as like that).

Turbo boost * Not available * available * available
 (turbo boost means clock speed can be upgraded)

Processing threads * Less (4 to 6) * medium (6-8) * max (12-16)

Cache memory * 8 MB * 9 MB * 12 MB

clock speed * 4.6 GHz * 4.7 GHz * 4.9 GHz

Kernal model * No * Yes * Yes

Ex:- using 3 address instruction

$$X = (P+Q) \times (R+S)$$

ADD R1, P, Q

ADD R2, R, S

MUL X, R1, R2

Ex:- using 2 address instruction

$$X = (P+Q) \times (R+S)$$

MOV R1, P

ADD R1, Q

MOV R2, R

ADD R2, S

MUL R1, R2

MOV X, R1

Ex:- one address instruction:-

LOAD P

LOAD R STORE X

ADD Q

ADD S

STORE T

MUL T

Ex:- RISC Instructions

$$X = (P+Q) \times (R+S)$$

LOAD R1, P

MUL R1, R1, R2

LOAD R2, Q

STORE X, R1

LOAD R3, R

LOAD R4, S

ADD R1, R1, R2

ADD R3, R3, R4

RISC and CISC Processors

RISC stands for **Reduced Instruction Set Computer** and
CISC stands for **Complex Instruction Set Computer**.

There are two approaches of the design of the control unit of a microprocessor i.e.-

- ✓ Hardware approach and
- ✓ Software approach.

RISC Processors It is known as Reduced Instruction Set Computer. It is a type of microprocessor that has a limited number of instructions. They can execute their instructions very fast because instructions are very small and simple.

RISC chips require fewer transistors which make them cheaper to design and produce. In RISC, the instruction set contains simple and basic instructions from which more complex instruction can be produced. Most instructions complete in one cycle, which allows the processor to handle many instructions at same time.

In this instructions are register based and data transfer takes place from register to register.

Examples of RISC processors are:

- ✓ DEC's Alpha 21064, 21164 and 21264 processors;
- ✓ SUN's SPARC and ULTRA SPARC;
- ✓ PowerPC processors etc.

CISC Processors

- ✓ It is known as Complex Instruction Set Computer.
- ✓ It was first developed by Intel.
- ✓ It contains large number of complex instructions.
- ✓ In this instructions are not register based.
- ✓ Instructions cannot be completed in one machine cycle.
- ✓ Data transfer is from memory to memory.
- ✓ Micro programmed control unit is found in CISC.
- ✓ Also they have variable instruction formats.

Examples of CISC processors are:

- ✓ Intel 386, 486;
- ✓ Pentium Pro, Pentium, Pentium II, Pentium III, Pentium 4;
- ✓ Motorola's 68000, 68020, 68030, 68040, etc.

RISC (Software)	CISC (Hardware)
Simple instruction set	Complex instruction set
Consists of Large number of registers.	Less number of registers
Larger Program	Smaller program
Simple processor circuitry (small number of transistors)	Complex processor circuitry (more number of transistors)
More RAM usage	Little Ram usage
Simple addressing modes	Variety of addressing modes
Fixed length instructions	Variable length instructions

8085 Microprocessor

8085 is pronounced as "eighty-eighty-five" microprocessor. It is an 8-bit microprocessor designed by Intel in 1977 using NMOS technology.

It has the following configuration –

- ✓ 8-bit data bus
- ✓ 16-bit address bus, which can address upto 64KB
- ✓ A 16-bit program counter
- ✓ A 16-bit stack pointer
- ✓ Six 8-bit registers arranged in pairs: BC, DE, HL
- ✓ Requires +5V supply to operate at 3.2 MHz single phase clock
- ✓ It is used in washing machines, microwave ovens, mobile phones, etc.

8085 Microprocessor – Functional Units

8085 consists of the following functional units –

✓ Accumulator

It is an 8-bit register used to perform arithmetic, logical, I/O & LOAD/STORE operations. It is connected to internal data bus & ALU.

✓ Arithmetic and logic unit

As the name suggests, it performs arithmetic and logical operations like Addition, Subtraction, AND, OR, etc. on 8-bit data.

✓ General purpose register

There are 6 general purpose registers in 8085 processor, i.e. B, C, D, E, H & L. Each register can hold 8-bit data.

These registers can work in pair to hold 16-bit data and their pairing combination is like B-C, D-E & H-L.

✓ Program counter

It is a 16-bit register used to store the memory address location of the next instruction to be executed. Microprocessor increments the program whenever an instruction is being executed, so that the program counter points to the memory address of the next instruction that is going to be executed.

✓ Stack pointer

It is also a 16-bit register works like stack, which is always incremented/decremented by 2 during push & pop operations.

Temporary register

It is an 8-bit register, which holds the temporary data of arithmetic and logical operations.

Flag register

It is an 8-bit register having five 1-bit flip-flops, which holds either 0 or 1 depending upon the result stored in the accumulator.

These are the set of 5 flip-flops –

- ✓ Sign (S)
- ✓ Zero (Z)
- ✓ Auxiliary Carry (AC)
- ✓ Parity (P)
- ✓ Carry (C)
- ✓

Data and Address Bus

- ✓ The Intel 8085 is an 8-bit microprocessor. Its data bus is 8-bit wide and therefore, 8 bits of data can be transmitted in parallel from or to the microprocessor.
- ✓ The Intel 8085 requires an address bus of 16-bit wide as the memory addresses are of 16-bits.
- ✓ The 8 most significant bits of the address are transmitted by the address bus, A-bus (pins A_8 ? A_{15}).
- ✓ The 8 least significant bits of the address are transmitted by data/address bus, AD-bus (pins AD_0 ? AD_7).

✎ Which one is the first 64-bit operating system? [SBL&JBL(O-IT)-2020]

- a) Windows Vista b) Mac c) Wom d) Spyware

Ans: b

Computer Architecture Topics

MACHINE INSTRUCTIONS

The operation of the CPU is determined by the instructions it executes. These instructions are called machine instructions or computer instructions.

- ✎ The collection of different instructions that the CPU can execute is referred to as the CPU's instruction set.

Elements of Machine Instructions

Each instruction must contain the information required by the processor for execution. The elements of a machine instruction are

- ✓ Operation code: Specifies the operation
- ✓ Source operand reference: Inputs for operation.
- ✓ Result operand reference
- ✓ Next instruction reference

Instruction Types

Number of addresses

Most of the instructions have one, two or three operand addresses, with the address of the next instruction being implicit.

- i. **3-address instructions:** Computers with 3-address instruction formats can use each address field to specify either a processor register or a memory operand.

Example: 3-address instruction format for the evaluation of $X = (P+Q) \times (R+S)$ is

ADD R1, P, Q

ADD R2, R, S

MUL X, R1, R2.

Here R1, R2, are processor registers.

Advantage: Shorter programs when evaluating arithmetic expressions.

Disadvantage: The binary coded instructions required too many bits to specify three addresses.

- ii. **2-address instructions:** These are the most common in commercial computers. Each address field can specify either a processor register or a memory word.

Example: For evaluating $X = (P+Q) \times (R+S)$,

The 2-address instructions are

MOV R1, P

ADD R1, Q

MOV R2, R

ADD R2, S
 MUL R1, R2
 MOV X, R1

(The first symbol of instruction is both source and Destination)

iii. **One-address instructions:** Use an implied accumulator (AC) register for all data manipulations

Example: 1-address instructions to evaluate $R = (P+Q) \times (R+S)$

LOAD P	ADD S
ADD Q	MUL T
STORE T	STORE X
LOAD R	

Here, T is a temporary memory location required to store the intermediate result.

iv. **RISC instructions:** The instruction set of a reduced instruction set computer (RISC) processor is restricted to the use of load and store instruction when communicating between memory and CPU. All other instructions are executed within the register of the CPU without referring to memory.

Example: RISC instruction to evaluate,

$X = (P+Q) \times (R + S)$	ADD R1, R1, R2
LOAD R1, P	ADD R3, R3, R4
LOAD R2, Q	MUL R1, R1, R3
LOAD R3, R	STORE X, R1
LOAD R4, S	

ADDRESSING MODES

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.

Computers use addressing mode techniques for the purpose of accommodating the following provisions:

- ✓ Facilitates pointers to memory
- ✓ Facilitates counters for loop control
- ✓ Facilitates indexing of data
- ✓ Facilitates program relocation.
- ✓ Reduce the number of bits in the address field of the instruction.

The most common addressing techniques are

- i. Implied mode
- ii. Immediate mode
- iii. Direct mode
- iv. Indirect mode
- v. Register mode
- vi. Register Indirect mode
- vii. Auto-increment or Auto-decrement mode
- viii. Displacement mode
 1. PC relative mode
 2. Indexed mode

3. Base register mode

Addressing modes in 8085 microprocessor

The way of specifying data to be operated by an instruction is called addressing mode.

Types of addressing modes –

In 8085 microprocessor there are 5 types of addressing modes:

1. Immediate Addressing Mode –

The operand is specified in the instruction itself. In immediate addressing mode the source operand is always data. If the data is 8-bit, then the instruction will be of 2 bytes, if the data is of 16-bit then the instruction will be of 3 bytes.



Examples:

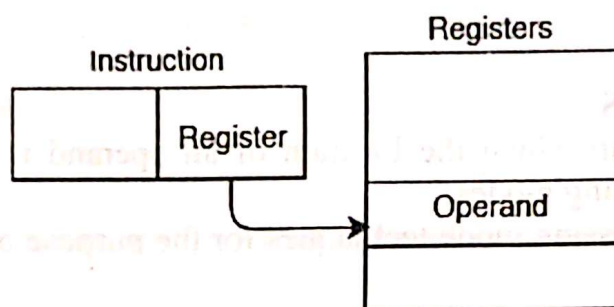
Move B 45 (move the data 45H immediately to register B)

Load H 3050 (load the H-L pair with the operand 3050H immediately)

JMP address (jump to the operand address immediately)

2. Register Addressing Mode –

In register addressing mode, the data to be operated is available inside the register(s) and register(s) is (are) operands. Therefore the operation is performed within various registers of the microprocessor.



Examples:

MOV A, B (move the contents of register B to register A)

ADD B (add contents of registers A and B and store the result in register A)

INR A (increment the contents of register A by one)

3. Direct Addressing Mode –

In direct addressing mode, the data to be operated is available inside a memory location and that memory location is directly specified as an operand. The operand is directly available in the instruction itself.

Examples:

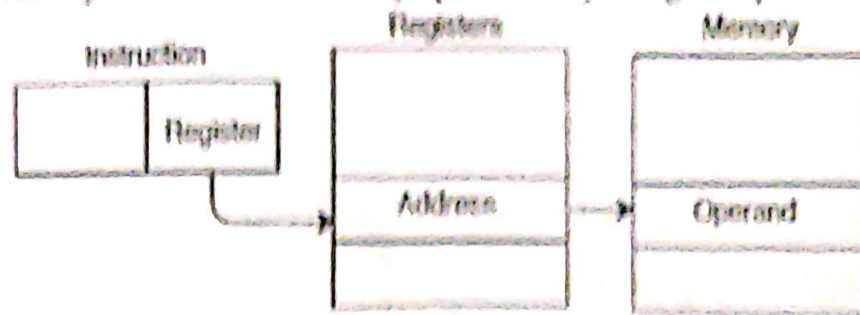
LDA 2050 (load the contents of memory location into accumulator A)

LHLD address (load contents of 16-bit memory location into H-L register pair)

IN 35 (read the data from port whose address is 01)

4. Register Indirect Addressing Mode –

IN register indirect addressing mode, the data to be operated is available inside a memory location and that memory location is indirectly specified by a register pair.



Examples:

MOV A, M (move the contents of the memory location pointed by the H-L pair to the accumulator)

LDAX B (move contents of B-C register to the accumulator)

LXIH 9570 (load immediate the H-L pair with the address of the location 9570)

5. Implied/Implicit Addressing Mode –

In implied/implicit addressing mode the operand is hidden and the data to be operated is available in the instruction itself.

Examples:

CMA (finds and stores the 1's complement of the contents of accumulator A in A)

RRC (rotate accumulator A right by one bit)

RLC (rotate accumulator A left by one bit)

Intel 8086 Microprocessor

8086 Microprocessor is an enhanced version of 8085 Microprocessor that was designed by Intel in 1976. It is a 16-bit Microprocessor having 20 address lines and 16 data lines that provides up to 1MB storage. It consists of powerful instruction set, which provides operations like multiplication and division easily.

It supports two modes of operation, i.e. Maximum mode and Minimum mode. Maximum mode is suitable for system having multiple processors and Minimum mode is suitable for system having a single processor.

Features of 8086

The most prominent features of an 8086 microprocessor are as follows –

- ✓ It has an instruction queue, which is capable of storing six instruction bytes from the memory resulting in faster processing.
- ✓ It was the first 16-bit processor having 16-bit ALU, 16-bit registers, internal data bus, and 16-bit external data bus resulting in faster processing.
- ✓ It is available in 3 versions based on the frequency of operation –
 - ✓ 8086 → 5MHz
 - ✓ 8086-2 → 8MHz
 - ✓ (c)8086-1 → 10 MHz

- ✓ It uses two stages of pipelining, i.e. Fetch Stage and Execute Stage, which improves performance.
- ✓ Fetch stage can prefetch up to 6 bytes of instructions and stores them in the queue.
- ✓ Execute stage executes these instructions.
- ✓ It has 256 vectored interrupts.
- ✓ It consists of 29,000 transistors.

Difference between Intel 8085 and 8086 Microprocessor

Intel 8085 Microprocessor	Intel 8086 Microprocessor
It is an 8-bit microprocessor.	It is a 16-bit microprocessor.
It has a 16-bit address line.	It has a 20-bit address line.
It has a 8-bit data bus.	It has a 16-bit data bus.
The memory capacity is 64 KB.	The memory capacity is 1 MB.
The Clock speed of this microprocessor is 3 MHz.	The Clock speed of this microprocessor varies between 5, 8 and 10 MHz for different versions.
It has five flags.	It has nine flags.
8085 microprocessor does not support memory segmentation.	8086 microprocessor supports memory segmentation.
It does not support pipelining.	It supports pipelining.
It is accumulator based processor.	It is general purpose register based processor.
It has no minimum or maximum mode.	It has minimum and maximum modes.
In 8085, only one processor is used.	In 8086, more than one processor is used. An additional external processor can also be employed.
It contains less number of transistors compare to 8086 microprocessor. It contains about 6500 transistor.	It contains more number of transistors compare to 8085 microprocessor. It contains about 29000 in size.

Addressing modes of 8086 [BB AME-2017]

The way for which an operand is specified for an instruction in the accumulator, in a general purpose register or in memory location, is called **addressing mode**.

The 8086 microprocessors have 8 addressing modes. Two addressing modes have been provided for instructions which operate on register or immediate data.

These two addressing modes are:

Register Addressing: In register addressing, the operand is placed in one of the 16-bit or 8-bit general purpose registers.

Example

- ✓ MOV AX, CX
- ✓ ADD AL, BL
- ✓ ADD CX, DX

Immediate Addressing: In immediate addressing, the operand is specified in the instruction itself.

Example

- ✓ MOV AL, 35H
- ✓ MOV BX, 0301H
- ✓ MOV [0401], 3598H
- ✓ ADD AX, 4836H

The remaining 6 addressing modes specify the location of an operand which is placed in a memory.

These 6 addressing modes are:

Direct Addressing: In direct addressing mode, the operand's offset is given in the instruction as an 8-bit or 16-bit displacement element.

Example

- ✓ ADD AL, [0301]

The instruction adds the content of the offset address 0301 to AL. the operand is placed at the given offset (0301) within the data segment DS.

Register Indirect Addressing: The operand's offset is placed in any one of the registers BX, BP, SI or DI as specified in the instruction.

Example

- ✓ MOV AX, [BX]

It moves the contents of memory locations addressed by the register BX to the register AX.

Based Addressing: The operand's offset is the sum of an 8-bit or 16-bit displacement and the contents of the base register BX or BP. BX is used as base register for data segment, and the BP is used as a base register for stack segment.

Effective address (Offset) = [BX + 8-bit or 16-bit displacement].

Example

- ✓ `MOV AL, [BX+05]`; an example of 8-bit displacement.
- ✓ `MOV AL, [BX + 1346H]`; example of 16-bit displacement.

Indexed Addressing: The offset of an operand is the sum of the content of an index register SI or DI and an 8-bit or 16-bit displacement.

Offset (Effective Address) = [SI or DI + 8-bit or 16-bit displacement]

Example

- ✓ `MOV AX, [SI + 05]`; 8-bit displacement.
- ✓ `MOV AX, [SI + 1528H]`; 16-bit displacement.

Based Indexed Addressing: The offset of operand is the sum of the content of a base register BX or BP and an index register SI or DI.

Effective Address (Offset) = [BX or BP] + [SI or DI]

Here, BX is used for a base register for data segment, and BP is used as a base register for stack segment.

Example

- ✓ `ADD AX, [BX + SI]`
- ✓ `MOV CX, [BX + SI]`

Based Indexed with Displacement: In this mode of addressing, the operand's offset is given by:

Effective Address (Offset) = [BX or BP] + [SI or DI] + 8-bit or 16-bit displacement

Example

- ✓ `MOV AX, [BX + SI + 05]`; 8-bit displacement
- ✓ `MOV AX, [BX + SI + 1235H]`; 16-bit displacement

Practic Problems

1. The instruction, `MOV AX, 0005H` belongs to the address mode Ans.c
 - a) register
 - b) direct
 - c) immediate
 - d) register relative
2. The instruction, `MOV AX, 1234H` is an example of Ans.c
 - a) register addressing mode
 - b) direct addressing mode
 - c) immediate addressing mode
 - d) based indexed addressing mode
3. If the data is present in a register and it is referred using the particular register, then it is Ans.b
 - a) direct addressing mode
 - b) register addressing mode
 - c) indexed addressing mode
 - d) immediate addressing mode

4. The instruction, `MOV AX,[BX]` is an example of Ans. d
- direct addressing mode
 - register addressing mode
 - register relative addressing mode
 - register indirect addressing mode
5. The addressing mode, where you directly specify the operand value is Ans. a
- Immediate
 - Direct
 - Definite
 - Relative
6. In the following indexed addressing mode instruction, `MOV S(RI),LOC` the effective address is Ans. d
- $EA = S + R1$
 - $EA = R1$
 - $EA = [R1]$
 - $EA = S + [R1]$

Microcontroller Overview

A microcontroller is a small and low-cost microcomputer, which is designed to perform the specific tasks of embedded systems like displaying microwave's information, receiving remote signals, etc. The general microcontroller consists of the processor, the memory (RAM, ROM, EPROM), Serial ports, peripherals (timers, counters), etc.

Difference between Microprocessor and Microcontroller

Microprocessors	Microcontrollers
It is used for big applications.	It is used to execute a single task within an application.
Microprocessor is the heart of computer system.	It is the heart of the embedded system.
It is just a processor. Memory and I/O components have to be having to be connected externally.	Microcontroller contains external processor along with internal memory and I/O components.
Since I/O and memory connected externally, the circuit becomes large.	Since I/O and memory present internally, the circuit is small.
Can't be used in compact systems and hence inefficient.	Can be used in compact systems and microcontroller is an efficient technique.
Cost of entire system increases.	Cost of entire system is low.

4. Access an operand in data memory.
5. Write the result into a register.

Let us see a real life example that works on the concept of pipelined operation. Consider a water bottle packaging plant. Let there be 3 stages that a bottle should pass through, Inserting the bottle(I), Filling water in the bottle(F), and Sealing the bottle(S). Let us consider these stages as stage 1, stage 2 and stage 3 respectively. Let each stage take 1 minute to complete its operation. Now, in a non pipelined operation, a bottle is first inserted in the plant, after 1 minute it is moved to stage 2 where water is filled. Now, in stage 1 nothing is happening. Similarly, when the bottle moves to stage 3, both stage 1 and stage 2 are idle. But in pipelined operation, when the bottle is in stage 2, another bottle can be loaded at stage 1. Similarly, when the bottle is in stage 3, there can be one bottle each in stage 1 and stage 2. So, after each minute, we get a new bottle at the end of stage 3. Hence, the average time taken to manufacture 1 bottle is :

Without pipelining = $9/3$ minutes = 3m

I F S | | | | |

| | | I F S | |

| | | | I F S (9 minutes)

With pipelining = $5/3$ minutes = 1.67m

I F S | |

| I F S |

| | I F S (5 minutes)

Thus, pipelined operation increases the efficiency of a system.

Pipeline Hazards

There are situations in pipelining when the next instruction cannot execute in the following clock cycle. These events are called *hazards*, and there are three different types.

- ✓ Structural Hazards
- ✓ Data Hazards
- ✓ Control Hazards

Structural Hazards

The first hazard is called a structural hazard. It means that the hardware cannot support the combination of instructions that we want to execute in the same clock cycle. A structural hazard in the laundry room would occur if we used a washerdryer combination instead of a separate washer and dryer or if our roommate was busy doing something else and wouldn't put clothes away. Our carefully scheduled pipeline plans would then be foiled.

As we said above, the MIPS instruction set was designed to be pipelined, making it fairly easy for designers to avoid structural hazards when designing a pipeline. Suppose, however, that we had a single memory instead of two memories. If the pipeline had a fourth instruction, we would see that in the same clock cycle the first instruction is accessing data from memory while the fourth instruction is fetching an instruction from that same memory. Without two

microprocessor & Computer Org.
memories, our pipeline could have a structural hazard. So, in single word structural hazard when a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.

Data Hazards

Data hazards occur when the pipeline must be stalled because one step must wait for another to complete. In a computer pipeline, data hazards arise from the dependence of one instruction on an earlier one that is still in the pipeline. For example, suppose we have an add instruction followed immediately by a subtract instruction that uses the sum (\$s0):

add \$s0, \$t0, \$t1

sub \$t2, \$s0, \$t3

Without intervention, a data hazard could severely stall the pipeline. The add instruction doesn't write its result until the fifth stage, meaning that we would have to waste three clock cycles in the pipeline. So, in single word data hazard also called a pipeline data hazard when a planned instruction cannot execute in the proper clock cycle because data that is needed to execute the instruction is not yet available.

Solve: forwarding Also called bypassing. A method of resolving a data hazard by retrieving the missing data element from internal buffers rather than waiting for it to arrive from programmervisible registers or memory.

Control Hazards

The third type of hazard is called a control hazard, arising from the need to make a Decision based on the results of one instruction while others are executing.

Control hazard also called branch hazard. When the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.

Previous Year Questions:

1. Describe addressing mode of 8086 microprocessor. [BB AME-2017]
2. Explain the function of ALU and control unit of computer. [Multiple Ministry-(AP)2017]
3. What are the advantages of microcontroller over microprocessor? [ICT Ministry-(AME)2014]
4. What is CPU? Explain the functionalities of various part of CPU. [Ministry of Health-(AME)2014]
5. Describe different types of computer hazard. [Sonali/janata(oit)2014]
6. Write down the Computer architecture characteristics. Write down the stage of DLX pipelines. [BB(AME)-2019]

Model Test

1. In 8085 microprocessor the address bus is of _____ bits.
a) 4 b) 8 c) 16 d) 32
2. In 8085 microprocessor the data bus is of _____ bits.
a) 4 b) 8 c) 16 d) 32

3. In the architecture of 8085 microprocessor matches the following.

List-I

- A) Processing Unit
- B) Instruction Unit
- C) Storage and Interface Unit

List-II

- 1) Interrupt
- 2) General purpose register
- 3) ALU
- 4) Timing and Control

Codes:

	A	B	C
a)	4	1	2
b)	3	4	2
c)	2	3	1
d)	1	2	4

4. An n bit microprocessor has-
- a) n -bit program counter
 - b) n -bit address register
 - c) n -bit ALU
 - d) n -bit instruction register
5. The instruction, MOV AX, 1234H is an example of
- a) register addressing mode
 - b) direct addressing mode
 - c) immediate addressing mode
 - d) based indexed addressing mode
6. The instruction, MOV AX,[BX] is an example of
- a) direct addressing mode
 - b) register addressing mode
 - c) register relative addressing mode
 - d) register indirect addressing mode
7. The instruction, MOV AX, 0005H belongs to the address mode
- a) register
 - b) direct
 - c) immediate
 - d) register relative
8. The decoded instruction is stored in _____
- a) IR
 - b) PC
 - c) Registers
 - d) MDR
9. Floating point representation is used to store
- a) Boolean Value
 - b) Whole number
 - c) real integer
 - d) Integer
10. During the execution of a program which gets initialized first?
- a) MDR
 - b) IR
 - c) PC
 - d) MAR
11. _____ is used to store data in registers.
- a) D flip flop
 - b) JK flip flop
 - c) RS flip flop
 - d) None of the mentioned
12. Any condition that causes a processor to stall is called as _
- a) Hazard
 - b) Page fault

- c) _____ of the mentioned

13. Which memory device is generally made of semiconductors?
a) RAM b) Hard-disk
c) Floppy disk d) Cd disk
14. The ALU makes use of _____ to store the intermediate results.
a) Accumulators b) Registers
c) Heap d) Stack
15. _____ Bus structure is usually used to connect I/O devices.
a) Single bus b) Multiple bus
c) Star bus d) Rambus
16. To reduce the memory access time we generally make use of _____.
a) Heaps b) Higher capacity RAM's
c) SDRAM's d) Cache's
17. CPU fetches the instruction from memory according to value of _____.
a) Program counter b) status register
c) instruction register d) program status word
18. Physical connection between Microprocessor Memory and other parts is called—
a) Address bus b) Data Bus c) path d) Hub
19. The situation wherein the data of operands are not available is called _____.
a) Data hazard b) Stock
c) Deadlock d) Structural hazard
20. The I/O interface required to connect the I/O device to the bus consists of _____.
a) Address decoder and registers b) Control circuits
c) Address decoder, registers and Control circuits d) Only Control circuits

1	2	3	4	5
c	b	b	d	c
16	17	18	19	20
d	a	a	a	c